
Documentation

for

SharpEditor

Prepared by

Lionte Eduard-Iulian: 1310B

Gorea Sabin-Gabriel: 1310B

Mârț Sebastian: 1310B

Miron Victor: 1307A

May 19, 2026

Table of Contents

Table of Contents	ii
Revision History	ii
1. Introduction	1
1.1 Purpose	1
1.2 Document Conventions	1
1.3 Intended Audience and Reading Suggestions	1
1.4 Product Scope	1
1.5 References	2
2. Overall Description	2
2.1 Product Perspective	2
2.2 Product Functions	2
2.3 User Classes and Characteristics	3
2.4 Operating Environment	3
2.5 Design and Implementation Constraints	3
2.6 User Documentation	4
2.7 Assumptions and Dependencies	4
3. External Interface Requirements	5
3.1 User Interfaces	5
3.2 Hardware Interfaces	5
3.3 Software Interfaces	5
3.4 Communications Interfaces	5
4. System Features	5
4.1 Authentication	6
4.2 Document Management	7
4.3 Share Management	8
5. Other Nonfunctional Requirements	9
5.1 Performance Requirements	9
5.2 Safety Requirements	9
5.3 Security Requirements	9
5.4 Software Quality Attributes	9
5.5 Business Rules	9
6. Other Requirements	11
6.1 Database Requirements	11
6.2 Configurations Requirements	12
Appendix A: Glossary	12
Appendix B: UML Diagrams	13
B.1 Class Diagrams	13
B.2 Sequence Diagram	17
B.3 Activity Diagram	18
B.4 Use case Diagram	19
Appendix C: Significant Source Code	20
C.1 Memento	20

C.2 Memento	22
C.3 Unit Tests and results	25
Appendix D: Shared Tasks	34

Revision History

Name	Date	Reason For Changes	Version
Mârț Sebastian	19-05-2026	Initial Draft	1.0.0

1. Introduction

Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements for SharpEditor, a cross-platform collaborative document editing application. The document covers version 1.0 of the backend REST API and its integration points with the .NET MAUI frontend client.

SharpEditor enables authenticated users to create, edit, and share text documents with fine-grained permission control. This SRS is intended to serve as the authoritative reference for development, testing, and future maintenance of the system.

Document Conventions

This document follows the IEEE 830-1998 SRS standard structure. Requirements are identified using the prefix REQ- followed by a unique number (e.g., REQ-1). Priority levels used throughout this document are:

- High - essential for the system to function; must be delivered in v1.0
- Medium - important but the system can operate without it initially
- Low - desirable enhancement for future iterations

All endpoint paths are relative to the API base URL. HTTP methods are written in uppercase (GET, POST, PUT, DELETE). JSON field names appear in monospace notation.

Intended Audience and Reading Suggestions

This SRS is written for the following audiences:

- Developers - read all sections, focusing on Sections 3 and 4 for implementation details
- Testers - focus on Section 4 (System Features) and Section 5 (Non-functional Requirements) for test coverage
- Project managers - read Sections 1 and 2 for scope, and Section 5 for quality targets
- Future maintainers - use this document alongside the source code to understand architectural decisions

New readers are advised to start with the Overall Description (Section 2) to understand the product context before studying the specific requirements in Sections 3 and 4.

Product Scope

SharpEditor is a lightweight collaborative document management system. Its core purpose is to allow users to create and maintain versioned text documents, then share those documents with other registered users at defined permission levels (ReadOnly or ReadWrite).

The backend exposes a stateless REST API secured with JSON Web Tokens (JWT). It handles authentication, document storage and versioning, and permission management. The frontend (to be documented separately) is a .NET MAUI application that consumes this API via standard HTTP requests from Android, iOS, Windows, and macOS devices.

Key business goals:

- Provide a simple, reliable platform for shared note-taking and document collaboration
- Prevent data loss through optimistic-locking version control on document edits
- Enforce access control so owners retain full authority over their documents

2. Overall Description

Product Perspective

SharpEditor is a new, self-contained application. It does not replace any existing system. The product consists of two independent subsystems that communicate over HTTP:

- Backend API - an ASP.NET Core 9 application providing RESTful endpoints for authentication, document management, and share management. It persists data in a SQLite database via Entity Framework Core using the Repository and Unit of Work patterns.
- Frontend Client - a .NET MAUI cross-platform application (to be described in a future revision of this document) that consumes the backend API and runs natively on Android, iOS, Windows, and macOS.

The backend is the primary subject of this SRS. It is designed to be stateless; all session state is carried by the JWT Bearer token sent with each authenticated request.

Product Functions

At a high level, SharpEditor provides the following major function groups:

- Authentication — user registration and login with JWT-based session management
- Document Management — create, read, update, and delete documents owned by the authenticated user
- Shared Document Access — browse and interact with documents shared with the authenticated user
- Share Management — grant, modify, and revoke access to documents for other users

User Classes and Characteristics

SharpEditor has a single user class: the Registered User. All users interact with the system through the same set of API endpoints, but their effective capabilities on any given document are determined by their permission level:

- Owner — the user who created a document. Has full read, write, and sharing capabilities.

- ReadWrite collaborator — a user the Owner has explicitly granted write access to. Can view and edit the document content.
- ReadOnly collaborator — a user the Owner has explicitly granted read-only access. Can view but not modify the document.

No administrator or super-user role exists in v1.0.

Operating Environment

- Server OS: Any platform supported by .NET 9 (Linux, Windows, macOS)
- Database: SQLite (file-based, embedded; no external database server required in development)
- Runtime: .NET 9 SDK / ASP.NET Core 9
- Client: .NET MAUI application targeting Android 8.0+, iOS 14+, Windows 10+, and macOS 12+
- Network: Standard HTTPS

Design and Implementation Constraints

- Language: C# 13 / .NET 9 for the backend
- ORM: Entity Framework Core with SQLite provider; schema is managed via EF Core migrations
- Authentication: JWT Bearer tokens only; no OAuth, API keys, or session cookies
- Password storage: PBKDF2-SHA256 with 100,000 iterations and a random 16-byte salt (no plaintext passwords are stored)
- Versioning: Optimistic concurrency control via a monotonically increasing integer version field on documents
- Document format: Plain text (string) stored in the database; no binary attachments or rich formatting in v1.0, with a maximum of 500.000 characters per file

User Documentation

The following documentation will be provided with v1.0:

- This SRS document
- API endpoint summary (inline in this document, Section 4)
- Frontend usage guide

Assumptions and Dependencies

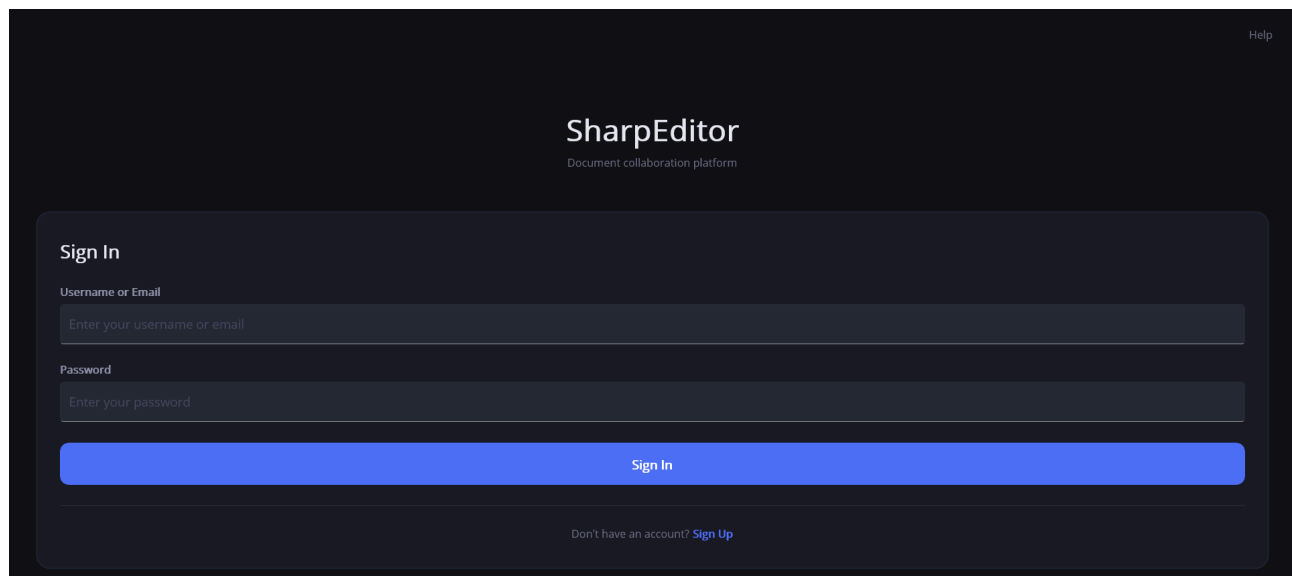
- It is assumed that every client of the API will include the Authorization: Bearer <token> header on all protected routes.
- The SQLite database file is stored on persistent disk; loss of the file means loss of all data (no automated backup in v1.0).
- The JWT secret key (Jwt:Secret in configuration) is kept confidential; compromise of this key allows token forgery.
- The system clock of the server is accurate; token expiry validation depends on it.
- EF Core migrations are applied automatically at application startup via `context.Database.Migrate()`.

3. External Interface Requirements

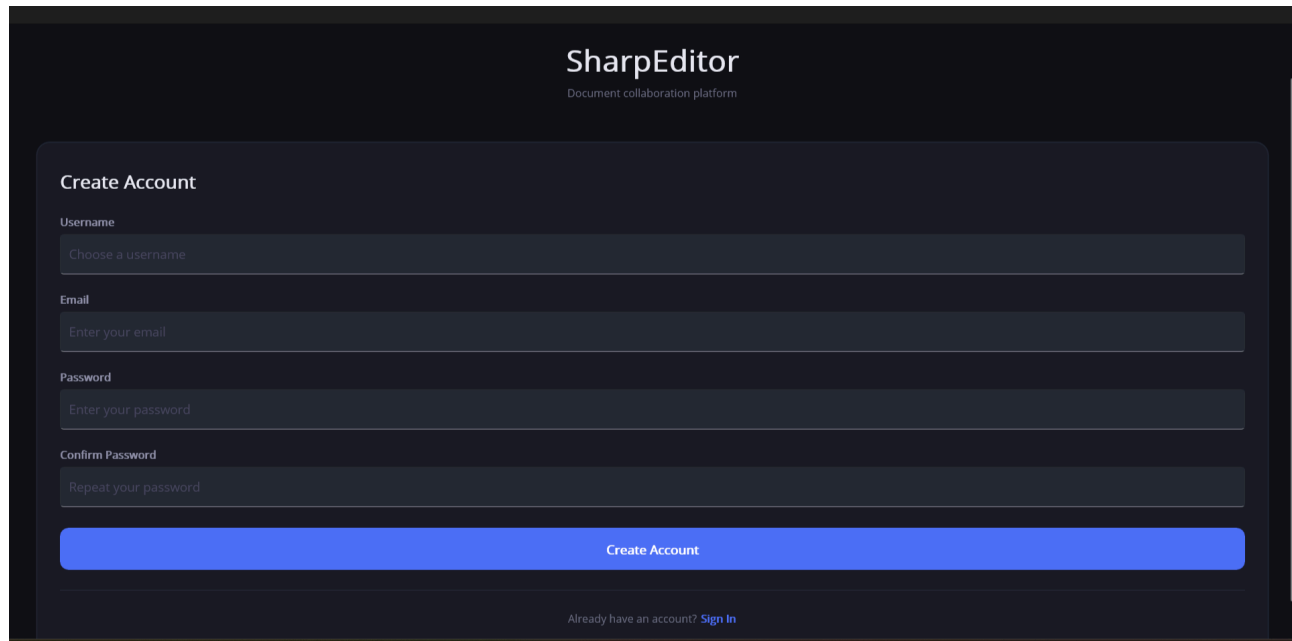
User Interfaces

The application provides a graphical user interface built with .NET MAUI, targeting Windows desktop. The interface follows a dark theme with a consistent color scheme across all screens.

Login Page: The first screen displayed on application launch. Contains two input fields (Username/Email and Password), a Sign In button, and a navigation link to the Register page. Displays inline error messages for invalid or empty inputs. A loading indicator is shown during authentication. A Help button is available in the top-right corner to open the user manual.

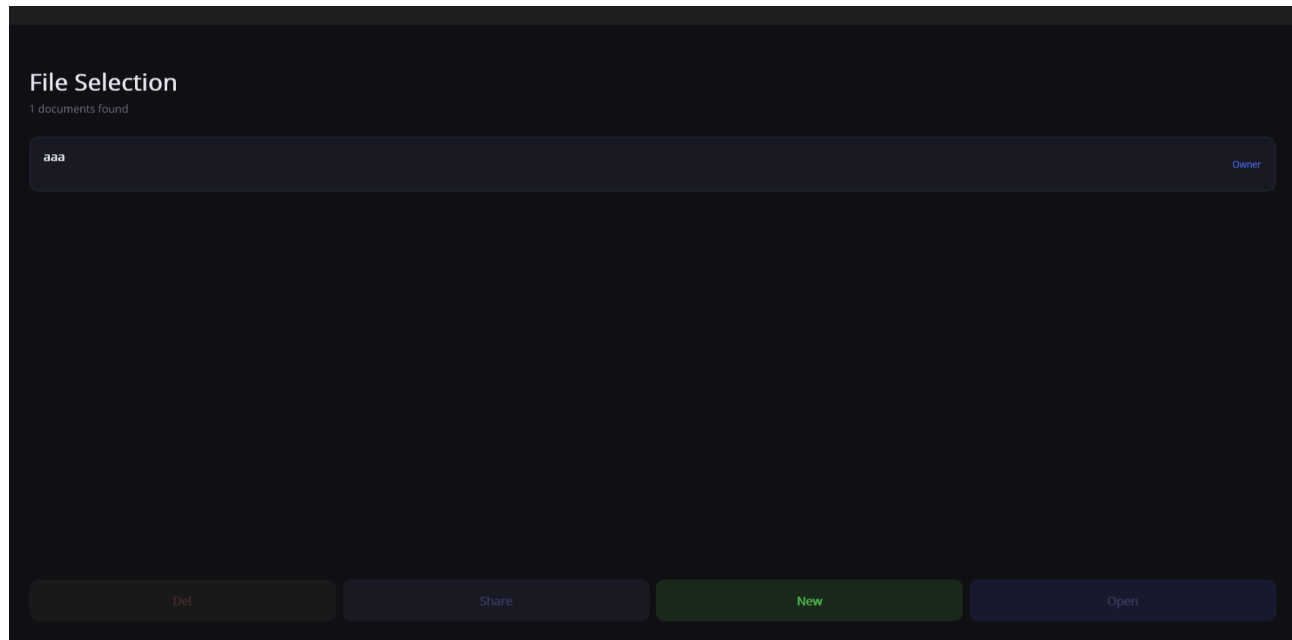


Register Page Accessible from the Login page. Contains four input fields (Username, Email, Password, Confirm Password), a Create Account button, and a navigation link back to the Login page. Performs local validation before submission: checks for empty fields, mismatched passwords, and minimum password length of 6 characters.

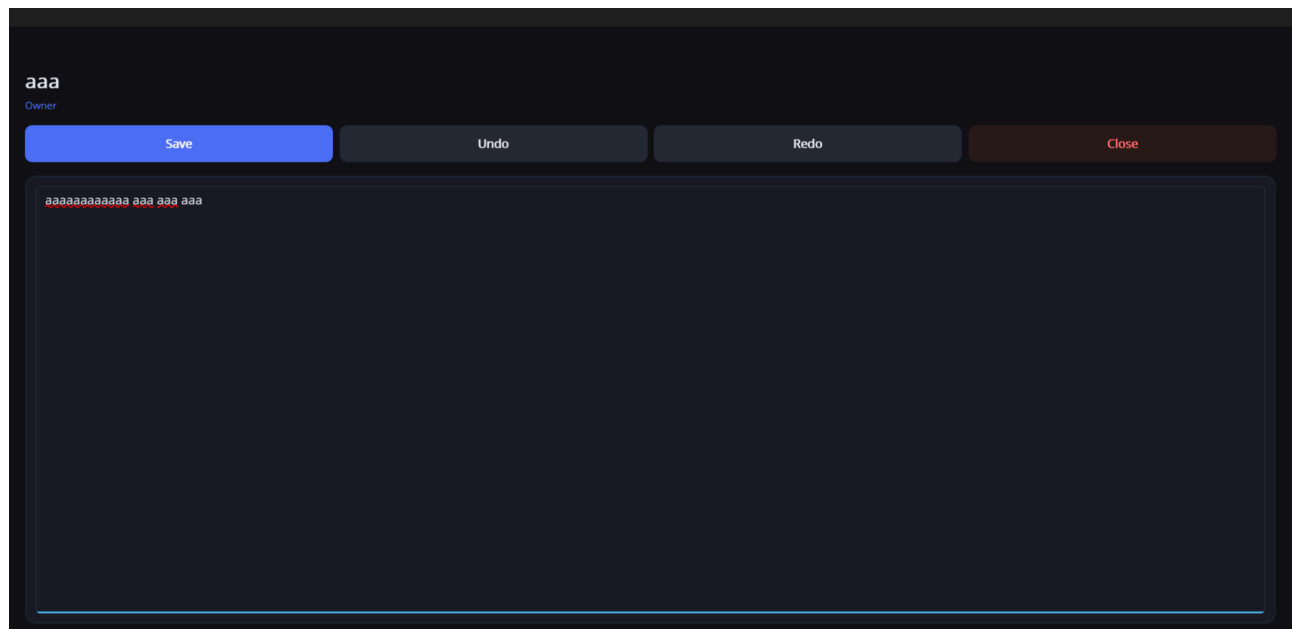


The screenshot shows the 'Create Account' page of the SharpEditor application. The page has a dark theme. At the top, the 'SharpEditor' logo is displayed with the tagline 'Document collaboration platform' below it. The main section is titled 'Create Account' and contains four input fields: 'Username' (placeholder: 'Choose a username'), 'Email' (placeholder: 'Enter your email'), 'Password' (placeholder: 'Enter your password'), and 'Confirm Password' (placeholder: 'Repeat your password'). Below these fields is a prominent blue 'Create Account' button. At the bottom of the form, there is a link that says 'Already have an account? Sign In'.

File Selection Page Displayed after successful login. Shows a list of all documents accessible to the authenticated user, including document title, last modified date, and permission level (Owner, ReadWrite, or ReadOnly). When a document is selected it is highlighted with a blue border. Four buttons are available at the bottom: New (create a new document), Open (open the selected document), Share (manage sharing for the selected document), and Delete (delete the selected document with confirmation dialog). The Open, Share and Delete buttons are disabled until a document is selected. Selecting a document and pressing the Share button opens a popup dialog. The dialog allows the Owner to add a new user by entering their user ID and selecting a permission level (ReadOnly or ReadWrite), and to remove an existing collaborator by selecting them and pressing Remove. The Share button is only active when a document is selected and is only functional for the document Owner



Read/Write File Page Opened when the user selects a document and presses Open, or creates a new document. Displays the document title and permission level at the top. Contains four action buttons: Save, Undo, Redo, and Close. The main area is a scrollable text editor. If the user has ReadOnly permission, the editor and Save/Undo/Redo buttons are disabled. Closing with unsaved changes triggers a confirmation dialog.



Hardware Interfaces

There are no direct hardware interface requirements. The application runs on commodity server hardware. The only hardware requirement is persistent disk storage for the SQLite database file.

Software Interfaces

The following software components are consumed by the backend:

Component	Version	Purpose
.NET / ASP.NET Core	9.x	Web framework and HTTP server
Entity Framework Core	9.x	ORM and database migration tool
Microsoft.EntityFrameworkCore.Sqlite	9.x	SQLite database provider for EF Core
Microsoft.AspNetCore.Authentication.JwtBearer	9.x	JWT Bearer authentication middleware
System.IdentityModel.Tokens.Jwt	Latest	JWT token generation and validation
System.Security.Cryptography	Built-in	PBKDF2 password hashing

Communications Interfaces

The API uses HTTP/1.1 and HTTP/2 over TCP. All API request and response bodies use the application/json content type. JWT tokens are transmitted in the Authorization HTTP request header using the Bearer scheme.

4. System Features

Authentication

4.1.1 Description and Priority

Provides user registration and login. New users create an account with a username, email address, and password. Existing users authenticate with email and password and receive a JWT for subsequent API calls.

Priority: High

4.1.2 Stimulus/Response Sequences

- User submits POST /auth/register with {Username, Email, Password} : system validates input, checks uniqueness, hashes password, creates user, returns 201 with confirmation message
- User submits POST /auth/login with {Email, Password} : system validates credentials, issues JWT, returns 200 with token
- Invalid or malformed input : 400 Bad Request with descriptive error message
- Duplicate email or username on registration : 409 Conflict
- Wrong password or unknown email on login : 401 Unauthorized

4.1.3 Functional Requirements

REQ-1: The system shall accept a registration request containing a non-empty Username, a valid email address, and a password of at least 6 characters.

REQ-2: The system shall reject registration if the email address is already in use (case-insensitive comparison after trimming and lowercasing).

REQ-3: The system shall reject registration if the username is already taken.

REQ-4: The system shall hash passwords using PBKDF2-SHA256 with 100,000 iterations and a random 16-byte salt before storing them.

REQ-5: On successful registration the system shall return HTTP 201 with a JSON body containing a confirmation message and no token.

REQ-6: The system shall authenticate a user by verifying the supplied password against the stored PBKDF2 hash using a constant-time comparison.

REQ-7: On successful login the system shall return HTTP 200 with a signed JWT containing the user ID (sub claim) and email, valid for the configured expiry period (default 60 minutes).

Document Management

4.1.1 Description and Priority

Allows authenticated users to create new documents, view documents they have access to, update document content (with version conflict detection), and delete documents.

Priority: High

4.1.2 Stimulus/Response Sequences

- POST /shared-docs with {Title, Content?} → creates document, auto-creates Owner share entry, returns 201 with full document detail
- GET /shared-docs → returns list of all documents the caller has access to (owned or shared), with title and permission
- GET /shared-docs/{id} → returns full document detail if caller has any share entry for that document

- PUT /shared-docs/{id} with {Content, Version} → updates content if caller has Owner or ReadWrite permission and Version matches the current server version; increments version; returns updated detail
- DELETE /shared-docs/{id} → if caller is Owner: deletes document and all share entries; if caller is collaborator: removes only their share entry

4.1.3 Functional Requirements

REQ-8: The system shall require an authenticated JWT to access any document endpoint.

REQ-9: On document creation, the system shall automatically create a UserDocument entry linking the creator to the document with SharePermission.Owner.

REQ-10: The system shall initialize new documents with Version = 1, CreatedAt = UtcNow, and UpdatedAt = UtcNow.

REQ-11: The system shall reject document creation if Title is missing or empty (400 Bad Request).

REQ-12: The system shall return 404 Not Found when the caller requests a document to which they have no share entry.

REQ-13: The system shall return 403 Forbidden when a ReadOnly collaborator attempts to update a document.

REQ-14: The system shall detect version conflicts: if the Version in the update request does not equal the current document version, the system shall return 409 Conflict.

REQ-15: On a successful update, the system shall increment the document Version by 1 and update UpdatedAt to UtcNow.

REQ-16: When the Owner deletes a document, the system shall remove all share entries for that document before deleting the document record.

REQ-17: When a non-Owner deletes their access, the system shall remove only their share entry; the document and other shares remain intact.

Share Management

4.3.1 Description and Priority

Enables document Owners to grant, view, modify, and revoke access for other registered users on a document-by-document basis.

Priority: High

4.3.2 Stimulus/Response Sequences

- GET /shared-docs/{id}/shares → returns list of collaborators (non-Owner shares) with userId, username, and permission; only Owner can call this

- POST /shared-docs/{id}/shares with {UserId, Permission} → adds a new share entry; only Owner can call this
- PUT /shared-docs/{id}/shares/{userId} with {Permission} → updates an existing share's permission level; only Owner can call this
- DELETE /shared-docs/{id}/shares/{userId} → removes a collaborator's access; only Owner can call this

4.3.3 Functional Requirements

REQ-18: Only a user with SharePermission.Owner on a document may view, add, modify, or revoke shares for that document. All other callers receive 403 Forbidden.

REQ-19: Valid permission values for shares are ReadOnly and ReadWrite. Any other value shall result in 400 Bad Request.

REQ-20: An Owner may not share a document with a user who already has a share entry for that document; the system shall return 409 Conflict.

REQ-21: The system shall return 404 Not Found when attempting to update or revoke a share for a user who has no existing share entry on that document.

REQ-22: The system shall return 404 Not Found when the target UserId does not correspond to any registered user.

REQ-23: The Owner's own share entry (SharePermission.Owner) shall not appear in the GET shares list; it is implicit.

5. Other Nonfunctional Requirements

Performance Requirements

All API endpoints shall respond within 500 ms under normal load (a single concurrent user on development hardware). The system shall support at least 50 concurrent users without significant degradation in response time for v1.0. JWT generation and validation shall complete in under 10 ms on the server side.

Safety Requirements

The system shall not expose raw exception stack traces or internal error messages to the client; EF Core migrations are applied automatically at startup. If migration fails, the application shall log the error and terminate rather than continue with an inconsistent database schema.

Security Requirements

- All passwords shall be stored as PBKDF2-SHA256 hashes with a unique random salt. Plaintext passwords shall never be logged or returned in any response.

- JWT tokens shall be signed with HMAC-SHA256 using a secret key configured outside source code (via appsettings or environment variables). The default expiry is 60 minutes with zero clock skew tolerance.
- The API shall validate the JWT issuer, audience, signing key, and lifetime on every protected request.
- The system shall use constant-time comparison when verifying password hashes to prevent timing attacks.

Software Quality Attributes

- Maintainability: The codebase follows a layered architecture (Controllers → Services → Repositories → Entities). Each layer depends only on abstractions (interfaces), enabling independent testing and replacement.
- Testability: The Unit of Work pattern exposes all repository operations through injectable interfaces, facilitating unit tests with mock repositories without a real database.
- Reliability: Optimistic concurrency control (document versioning) prevents silent data loss due to concurrent edits.
- Portability: The application runs on any platform supporting .NET 9. Switching from SQLite to another database (e.g., PostgreSQL) requires only a provider swap in ApplicationDbContext and connection string update.

Business Rules

- A document must always have exactly one Owner. Ownership cannot be transferred in v1.0.
- The Owner is the only party who can grant, modify, or revoke shares on their documents.
- A collaborator can remove themselves from a shared document by calling DELETE /shared-docs/{id}, which removes their share entry without affecting the document or other collaborators.
- Version numbers start at 1 and increment by 1 on every accepted update. Clients must supply the current version with each update to prove they have the latest state.

6. Other Requirements

6.1 Database Requirements

SharpEditor uses Entity Framework Core Code-First migrations to manage the database schema. The migration history is tracked in the `__EFMigrationsHistory` table inside the SQLite database file. Migrations are applied automatically at application startup via `context.Database.Migrate()`, ensuring the schema is always up to date without manual intervention.

New migrations must be generated using the EF Core CLI tool and committed to source control alongside the application code:

- `dotnet ef migrations add <MigrationName>` — generates a new migration based on model changes
- `dotnet ef database update` — applies pending migrations manually (for development use)

The SQLite database file path is defined by the `ConnectionStrings:DefaultConnection` key. In v1.0 the file is stored on local disk alongside the application. There is no automated backup mechanism; operators are responsible for backing up the database file in production environments. The maximum document content size is 500,000 characters per document, enforced at the service layer.

6.2 Configuration Requirements

All runtime configuration is read from `appsettings.json` or environment variable overrides at startup. The following keys are required; the application will fail to start or produce incorrect behaviour if any are missing or misconfigured:

- `ConnectionStrings:DefaultConnection` — SQLite connection string (e.g., `Data Source=app.db`). Required. The database file is created automatically if it does not exist.
- `Jwt:Secret` — HMAC-SHA256 signing key used to sign and verify JWT tokens. Required. Minimum recommended length is 32 characters. Must be kept secret; if compromised, all issued tokens can be forged.
- `Jwt:Issuer` — token issuer claim value. Default: `BackEnd`. Must match the value validated by the authentication middleware.
- `Jwt:Audience` — token audience claim value. Default: `BackEnd`. Must match the value validated by the authentication middleware.
- `Jwt:ExpiryMinutes` — access token lifetime in minutes. Default: 60. After expiry the client must re-authenticate via `POST /auth/login`.

On the frontend (.NET MAUI client), the backend base URL is hardcoded in `ApiService.cs` as the `BaseUrl` constant. When deploying to a different server, this value must be updated and the client recompiled. In a future version this should be made configurable via a settings screen or build-time configuration.

Appendix A: Glossary

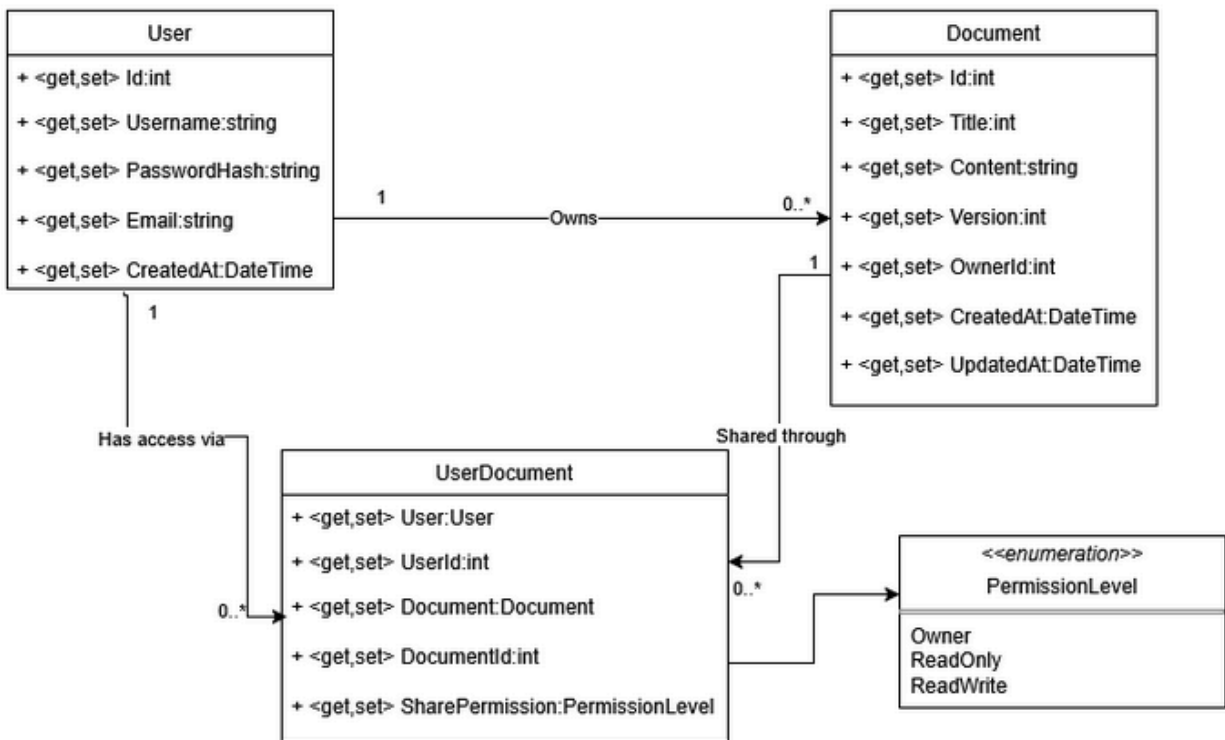
Term	Definition
JWT	JSON Web Token — a compact, URL-safe token format for securely transmitting claims between parties (RFC 7519).
Bearer Token	An HTTP authentication scheme where the client includes a JWT in the Authorization header: <code>Authorization: Bearer <token></code> .
PBKDF2	Password-Based Key Derivation Function 2 — a standard algorithm for hashing passwords with a salt and configurable iteration count.

EF Core	Entity Framework Core — a .NET ORM for relational databases.
ORM	Object-Relational Mapper — a library that maps database tables to .NET objects.
REST	Representational State Transfer — an architectural style for HTTP APIs using standard HTTP methods and stateless communication.
Unit of Work	A design pattern that groups database operations into a single transaction, committed or rolled back together.
Owner	The user who created a document; has full read, write, share-grant, share-revoke, and delete permissions.
ReadOnly	A share permission level granting a collaborator read-only access to a document.
ReadWrite	A share permission level granting a collaborator both read and write access to a document.
Version Conflict	A condition where the client's document version does not match the server's current version, indicating the document was modified since the client last fetched it.

Appendix B: UML Diagrams

B.1 Class diagrams

Entity Diagram:



Database integration diagram:

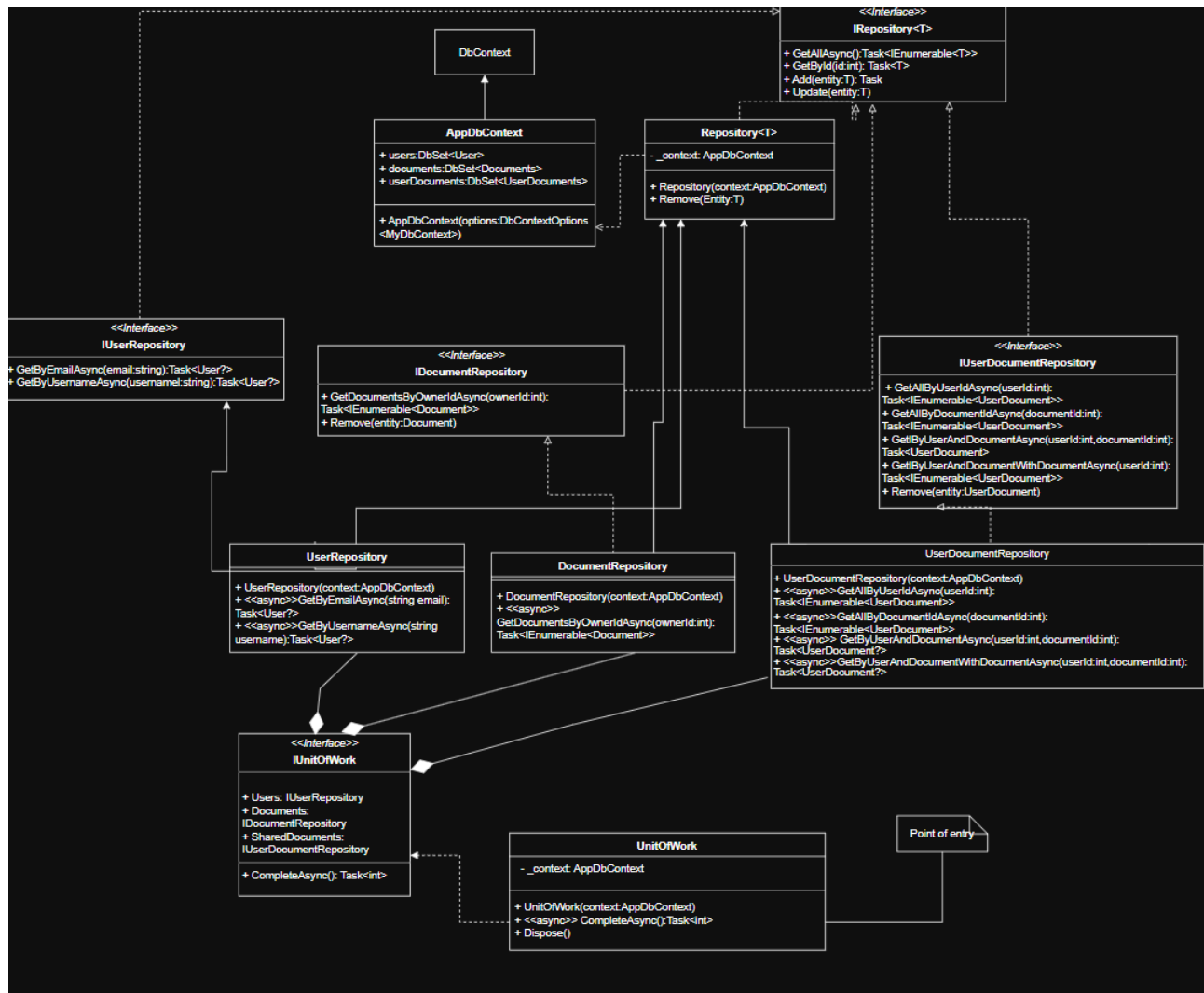
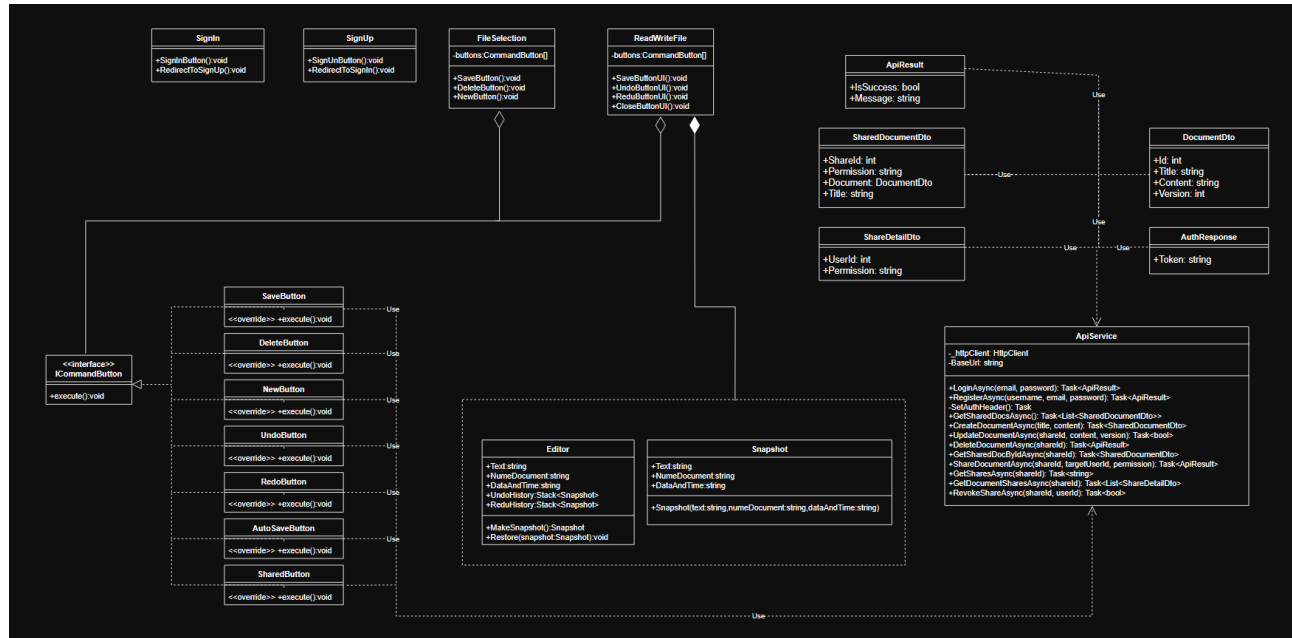
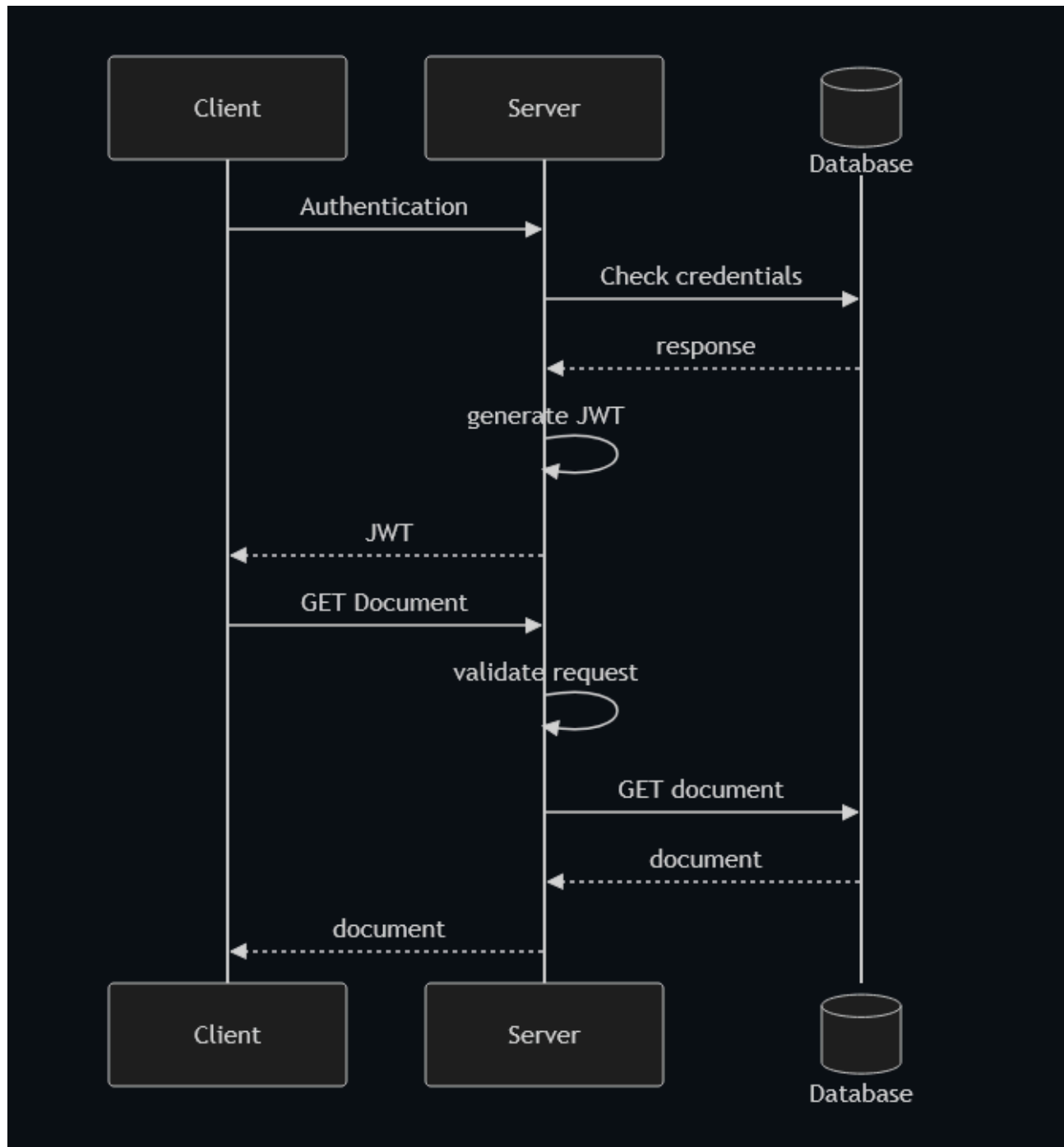


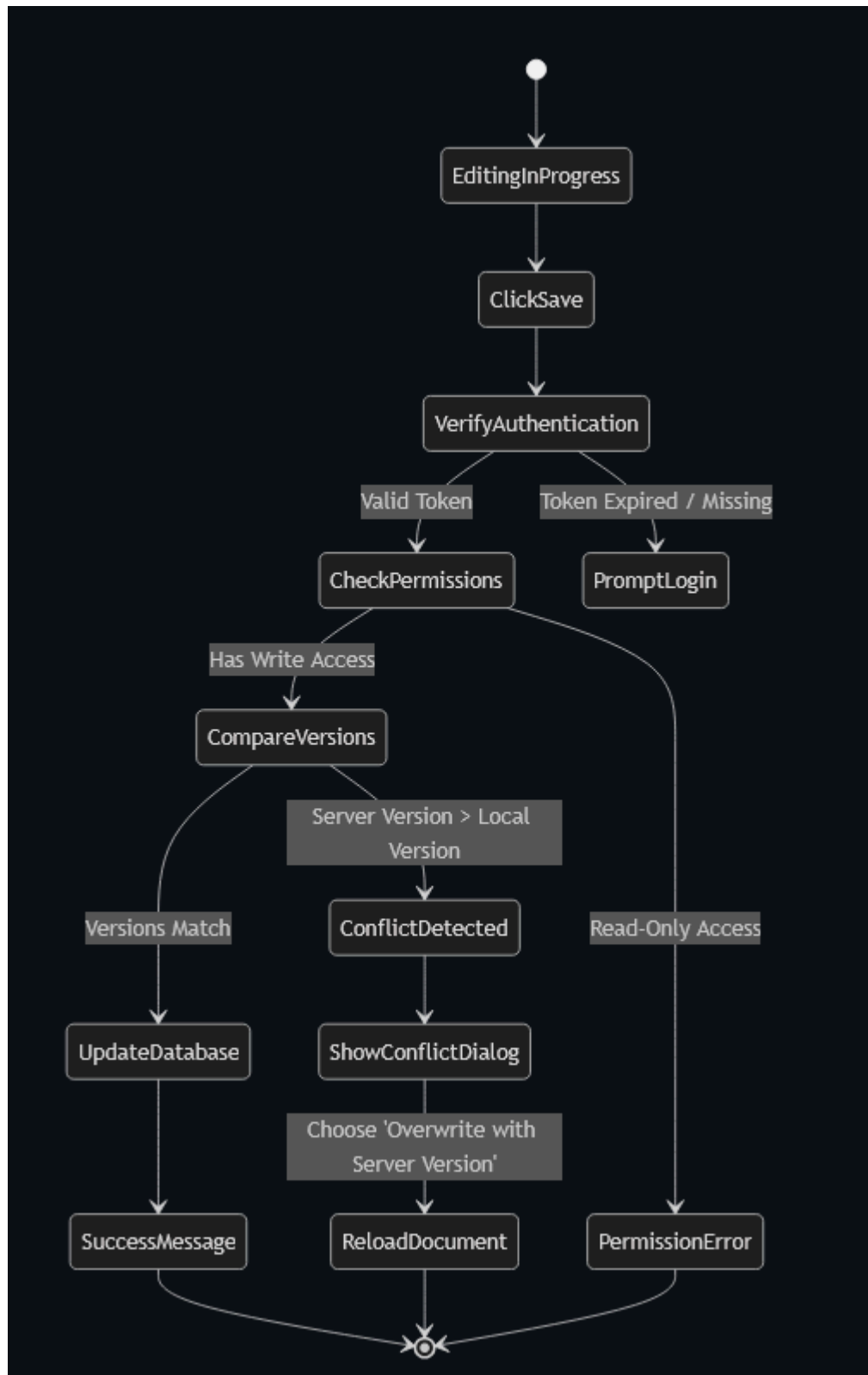
Diagrama Frontend:



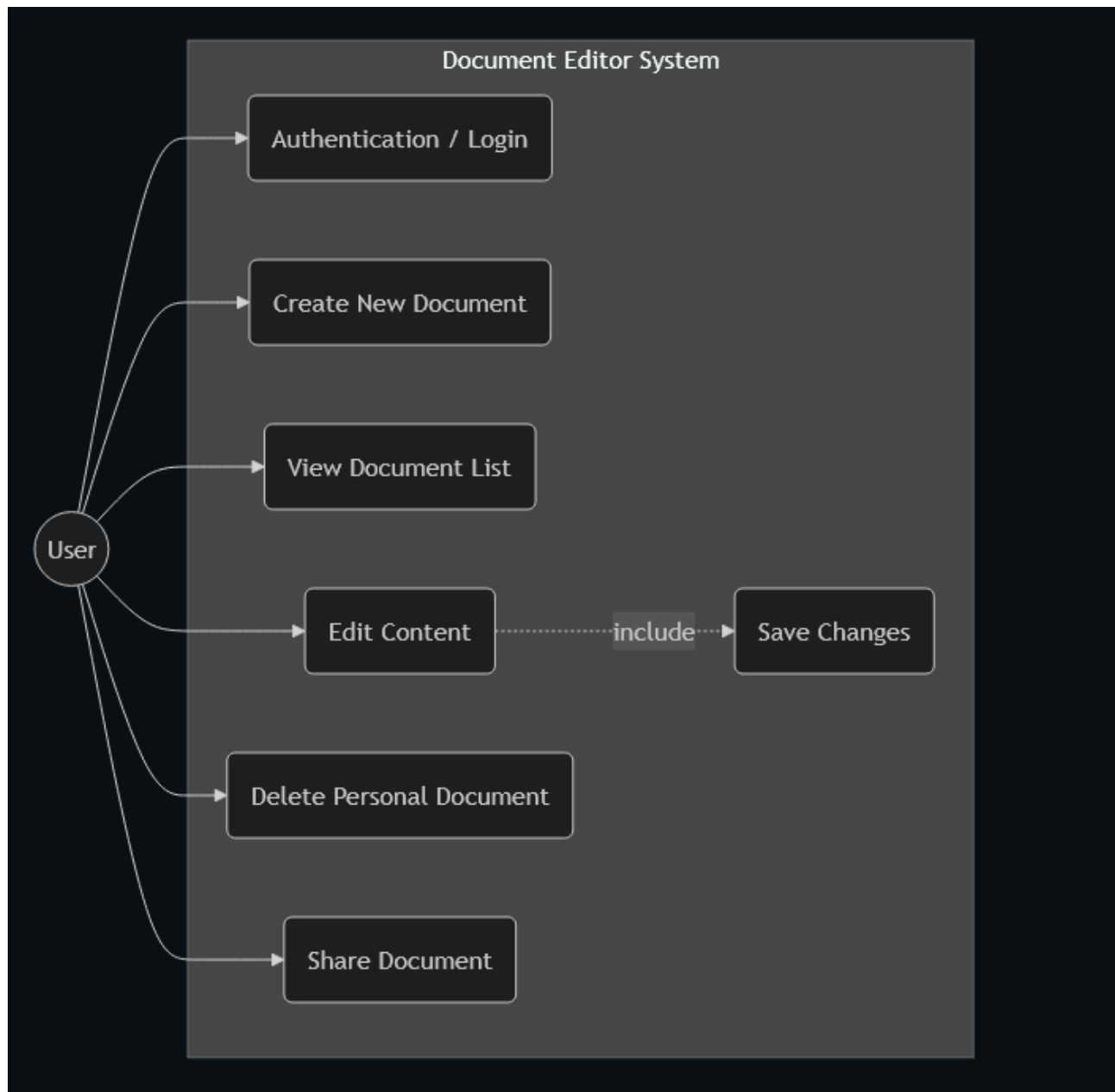
B.2 Sequence diagram



B.3 Activity diagram



B.4 Use case diagram



Appendix C: Significant Source Code

C.1 Memento

```
/*
 * Author: Lionte Eduard-Iulian
 * Description:
 * Represents the Memento object in the Memento pattern.
 * It captures and stores a point-in-time state of the editor, including text,
 * document title, and timestamp. It is designed to be immutable to ensure
 * the integrity of the undo/redo history.
 */

using System;
using System.Collections.Generic;
using System.Globalization;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace AplicatieUI.Logica.Memento
{
    internal class Snapshot
    {
        public string Text { get; }

        public string NumeDocument { get; }

        public string DataAndTime { get; }

        /// <summary>
        /// Initializes a new snapshot with the provided state data.
        /// </summary>
        public Snapshot(string text, string numeDocument = "", string dataAndTime =
        "")
        {
            this.Text = text;
            this.NumeDocument = numeDocument;
            this.DataAndTime = dataAndTime;
        }
    }
}
```

```
/*
 * Author: Lionte Eduard-Iulian
 * Description:
 * The Originator class in the Memento pattern.
 * It maintains the current editor state (text and metadata) and manages the
Undo/Redo
 * history stacks. It provides methods to capture the current state into a
Snapshot
 * and restore the editor's content from a previous snapshot.
 */

namespace AplicatieUI.Logica.Memento
{
    internal class Editor
    {
        public string Text { get; set; } = "";

        public string NumeDocument { get; set; } = "";

        public string DataAndTime { get; set; } = "";

        public Stack<Snapshot> UndoHistory { get; set; } = new();
        public Stack<Snapshot> RedoHistory { get; set; } = new();

        /// <summary>
        /// Creates a new snapshot of the current editor state.
        /// </summary>
        public Snapshot MakeSnapshot()
        {
            return new Snapshot(Text, NumeDocument, DataAndTime);
        }

        /// <summary>
        /// Restores the editor text from a given snapshot.
        /// </summary>
        public void Restore(Snapshot snapshot)
        {
            if (snapshot != null)
                Text = snapshot.Text;
        }
    }
}
```


C.2 Authentication

```
// =====
// File:          AuthController.cs
// Author:         Gorea Sabin Gabriel
// Description:    REST API controller that exposes authentication endpoints
//                 (registration and login). Delegates business logic to
//                 IAuthService and maps service-layer exceptions to the
//                 appropriate HTTP status codes.
// =====

using Core.business.DTOs;
using Core.business.Exceptions;
using Core.business.Services;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;

namespace Core.business.Controllers;

[ApiController]
[Route("auth")]
public class AuthController : ControllerBase
{
    // Service responsible for authentication logic (register / login)
    private readonly IAuthService _auth;

    // Inject IAuthService via constructor
    public AuthController(IAuthService auth) => _auth = auth;

    /// <summary>Register a new user.</summary>
    [HttpPost("register")]
    [ProducesResponseType(typeof(AuthResponse), StatusCodes.Status201Created)]
    [ProducesResponseType(StatusCodes.Status400BadRequest)]
    [ProducesResponseType(StatusCodes.Status409Conflict)]
    public async Task<IActionResult> Register([FromBody] RegisterRequest req)
    {
        try
        {
            // Delegate registration to the auth service
            var result = await _auth.RegisterAsync(req);

            // Return 201 Created with the generated auth response (e.g. JWT)
            return StatusCode(StatusCodes.Status201Created, result);
        }
        catch (ValidationException ex) { return BadRequest(new { error = ex.Message }); } // 400 - invalid input
        catch (ConflictException ex)   { return Conflict(new { error = ex.Message }); } // 409 - user already exists
    }

    /// <summary>Authenticate and receive a JWT.</summary>
    [HttpPost("login")]
    [ProducesResponseType(typeof(AuthResponse), StatusCodes.Status200OK)]
    [ProducesResponseType(StatusCodes.Status400BadRequest)]
    [ProducesResponseType(StatusCodes.Status401Unauthorized)]
}
```

```

public async Task<IActionResult> Login([FromBody] LoginRequest req)
{
    try
    {
        // Delegate credential verification to the auth service
        var result = await _auth.LoginAsync(req);

        // Return 200 OK with the auth response (e.g. JWT token)
        return Ok(result);
    }
    catch (ValidationException ex) { return BadRequest(new { error = ex.Message }); } // 400 - invalid input
    catch (UnauthorizedException ex) { return Unauthorized(new { error = ex.Message }); } // 401 - wrong credentials
}

// =====
// File:      AuthService.cs
// Author:    Gorea Sabin Gabriel
// Description: Defines the IAuthService interface and its implementation.
//             AuthService handles user registration and login logic:
//             validates input, checks uniqueness constraints, hashes
//             passwords, persists new users, and issues JWT tokens on
//             successful login.
// =====

using Core.business.DTOs;
using Core.business.Exceptions;
using Core.business.interfaces;
using Core.business.Security;
using Core.persistence.entities;

namespace Core.business.Services;

// Contract for authentication operations
public interface IAuthService
{
    Task<AuthResponse> RegisterAsync(RegisterRequest request);
    Task<AuthResponse> LoginAsync(LoginRequest request);
}

public class AuthService : IAuthService
{
    // Dependencies injected via constructor
    private readonly IUnitOfWork _uow; // Data access and transaction
management
    private readonly IPasswordHasher _hasher; // Password hashing and
verification
    private readonly IJwtService _jwt; // JWT token generation

    public AuthService(IUnitOfWork uow, IPasswordHasher hasher, IJwtService jwt)
    {
        _uow = uow;
        _hasher = hasher;
        _jwt = jwt;
    }

```

```

    }

    public async Task<AuthResponse> RegisterAsync(RegisterRequest req)
    {
        // Normalise email to lowercase and trim whitespace from both fields
        req = req with
        {
            Email      = req.Email.Trim().ToLowerInvariant(),
            Username    = req.Username.Trim()
        };

        // — validation —————
        if (string.IsNullOrEmpty(req.Username))
            throw new ValidationException("Username is required.");

        if (string.IsNullOrEmpty(req.Email) || !req.Email.Contains('@'))
            throw new ValidationException("A valid email is required.");

        if (string.IsNullOrEmpty(req.Password) || req.Password.Length < 6)
            throw new ValidationException("Password must be at least 6
characters.");

        // — uniqueness check —————
        // Ensure no existing account uses the same email
        var existingByEmail = await _uow.Users.GetByEmailAsync(req.Email);
        if (existingByEmail is not null)
            throw new ConflictException("Email already in use.");

        // Ensure no existing account uses the same username
        var existingByUsername = await
_uow.Users.GetByUsernameAsync(req.Username);
        if (existingByUsername is not null)
            throw new ConflictException("Username already taken.");

        // — create user —————
        var user = new User
        {
            Username      = req.Username,
            Email          = req.Email,
            PasswordHash  = _hasher.Hash(req.Password), // Store only the hashed
password
            CreatedAt      = DateTime.UtcNow,
        };

        await _uow.Users.AddAsync(user);
        await _uow.CompleteAsync(); // Persist changes to the database

        // Registration does not issue a token; return a confirmation message only
        return new AuthResponse(Token: null, Message: "User registered
successfully.");
    }

    public async Task<AuthResponse> LoginAsync(LoginRequest req)
    {
        // Ensure both fields are provided before querying the database
        if (string.IsNullOrEmpty(req.Email) ||
string.IsNullOrEmpty(req.Password))

```

```

        throw new ValidationException("Email and password are required.");

        // Normalise email to match the format used during registration
        var cleanEmail = req.Email.Trim().ToLowerInvariant();

        var user = await _uow.Users.GetByEmailAsync(cleanEmail);

        // Reject if user not found or if the password does not match the stored
hash    if (user is null || !_hasher.Verify(req.Password, user.PasswordHash))
        throw new UnauthorizedException("Invalid credentials.");

        // Issue and return a JWT token for the authenticated user
        return new AuthResponse(_jwt.GenerateToken(user.Id, user.Email));
    }
}

```

C.3 Unit tests and results

```

using Core.business.DTOs;
using Core.business.Exceptions;
using Core.business.interfaces;
using Core.business.Security;
using Core.business.Services;
using Core.persistance.entities;
using Moq;

namespace BackEnd.Tests;

public class AuthServiceTests
{
    private readonly Mock<IUnitOfWork> _uow = new();
    private readonly Mock<IUserRepository> _users = new();
    private readonly Mock<IPasswordHasher> _hasher = new();
    private readonly Mock<IJwtService> _jwt = new();
    private readonly IAuthService _sut;

    public AuthServiceTests()
    {
        _uow.Setup(u => u.Users).Returns(_users.Object);
        _sut = new AuthService(_uow.Object, _hasher.Object, _jwt.Object);
    }

    // — Register

    [Fact]
    public async Task Register_ValidRequest_ReturnsSuccessMessage()
    {

```

```

        _users.Setup(u =>
u.GetByEmailAsync(It.IsAny<string>())).ReturnsAsync((User?)null);
        _users.Setup(u =>
u.GetByUsernameAsync(It.IsAny<string>())).ReturnsAsync((User?)null);
        _hasher.Setup(h => h.Hash(It.IsAny<string>())).Returns("hashed");

        var result = await _sut.RegisterAsync(new RegisterRequest("alice",
"alice@example.com", "password123"));

        Assert.Null(result.Token);
        Assert.Contains("registered", result.Message,
StringComparison.OrdinalIgnoreCase);
        _uow.Verify(u => u.CompleteAsync(), Times.Once);
    }

    [Theory]
    [InlineData("", "alice@example.com", "password123")] // empty username
    [InlineData("alice", "not-an-email", "password123")] // invalid email
    [InlineData("alice", "alice@example.com", "123")] // short password
    public async Task Register_InvalidInput_ThrowsValidationException(
        string username, string email, string password)
    {
        await Assert.ThrowsAsync<ValidationException>(
            () => _sut.RegisterAsync(new RegisterRequest(username, email,
password)));
    }

    [Fact]
    public async Task Register_DuplicateEmail_ThrowsConflictException()
    {
        _users.Setup(u => u.GetByEmailAsync("alice@example.com"))
            .ReturnsAsync(new User { Email = "alice@example.com" });

        await Assert.ThrowsAsync<ConflictException>(
            () => _sut.RegisterAsync(new RegisterRequest("alice",
"alice@example.com", "password123")));
    }

    [Fact]
    public async Task Register_DuplicateUsername_ThrowsConflictException()
    {
        _users.Setup(u =>
u.GetByEmailAsync(It.IsAny<string>())).ReturnsAsync((User?)null);
        _users.Setup(u => u.GetByUsernameAsync("alice"))
            .ReturnsAsync(new User { Username = "alice" });

        await Assert.ThrowsAsync<ConflictException>(
            () => _sut.RegisterAsync(new RegisterRequest("alice",
"new@example.com", "password123")));
    }

    [Fact]
    public async Task Register_NormalizesEmailToLowercase()
    {
        _users.Setup(u =>
u.GetByEmailAsync("alice@example.com")).ReturnsAsync((User?)null);

```

```

        _users.Setup(u =>
u.GetByUsernameAsync(It.IsAny<string>())) .ReturnsAsync((User?)null);
        _hasher.Setup(h => h.Hash(It.IsAny<string>())) .Returns("hashed");

        await _sut.RegisterAsync(new RegisterRequest("alice",
"ALICE@EXAMPLE.COM", "password123"));

        _users.Verify(u => u.GetByEmailAsync("alice@example.com"), Times.Once);
    }

    // — Login

```

```

[Fact]
public async Task Login_ValidCredentials_ReturnsToken()
{
    var user = new User { Id = 1, Email = "alice@example.com", PasswordHash
= "hashed" };
    _users.Setup(u =>
u.GetByEmailAsync("alice@example.com")) .ReturnsAsync(user);
    _hasher.Setup(h => h.Verify("password123", "hashed")) .Returns(true);
    _jwt.Setup(j => j.GenerateToken(1,
"alice@example.com")) .Returns("jwt-token");

    var result = await _sut.LoginAsync(new LoginRequest("alice@example.com",
"password123"));

    Assert.Equal("jwt-token", result.Token);
}

[Fact]
public async Task Login_UnknownEmail_ThrowsUnauthorizedException()
{
    _users.Setup(u =>
u.GetByEmailAsync(It.IsAny<string>())) .ReturnsAsync((User?)null);

    await Assert.ThrowsAsync<UnauthorizedException>(
        () => _sut.LoginAsync(new LoginRequest("x@x.com", "password123")));
}

[Fact]
public async Task Login_WrongPassword_ThrowsUnauthorizedException()
{
    var user = new User { Id = 1, Email = "alice@example.com", PasswordHash
= "hashed" };
    _users.Setup(u =>
u.GetByEmailAsync("alice@example.com")) .ReturnsAsync(user);
    _hasher.Setup(h => h.Verify("wrong", "hashed")) .Returns(false);

    await Assert.ThrowsAsync<UnauthorizedException>(
        () => _sut.LoginAsync(new LoginRequest("alice@example.com",
"wrong")));
}

[Theory]
[InlineData("", "password123")]
[InlineData("alice@example.com", "")]

```

```

    public async Task Login_MissingFields_ThrowsValidationException(string
email, string password)
    {
        await Assert.ThrowsAsync<ValidationException>(
            () => _sut.LoginAsync(new LoginRequest(email, password)));
    }
}

using Core.business.DTOs;
using Core.business.Exceptions;
using Core.business.interfaces;
using Core.business.Services;
using Core.persistence.entities;
using Moq;

namespace BackEnd.Tests;

public class DocumentServiceTests
{
    private readonly Mock<IUnitOfWork> _uow = new();
    private readonly Mock<IDocumentRepository> _docs = new();
    private readonly Mock<IUserDocumentRepository> _shares = new();
    private readonly Mock<IUserRepository> _users = new();
    private readonly IDocumentService _sut;

    // Re-usable fixture data
    private static Document MakeDocument(int id = 1, int ownerId = 10) => new()
    {
        Id = id,
        OwnerId = ownerId,
        Title = "Doc",
        Content = "Hello",
        Version = 1,
        CreatedAt = DateTime.UtcNow,
        UpdatedAt = DateTime.UtcNow,
    };

    private static UserDocument MakeShare(
        int userId, int docId, SharePermission perm, Document? doc = null) =>
new()
    {
        UserId = userId,
        DocumentId = docId,
        PermissionLevel = perm,
        Document = doc ?? MakeDocument(docId),
    };

    public DocumentServiceTests()
    {
        _uow.Setup(u => u.Documents).Returns(_docs.Object);
        _uow.Setup(u => u.SharedDocuments).Returns(_shares.Object);
        _uow.Setup(u => u.Users).Returns(_users.Object);
        _sut = new DocumentService(_uow.Object);
    }
}

```

```
// — GetSharedDocs
```

```
[Fact]
public async Task GetSharedDocs_ReturnsSummariesForCaller()
{
    var share = MakeShare(10, 1, SharePermission.Owner);
    _shares.Setup(s => s.GetAllByUserIdAsync(10))
        .ReturnsAsync(new[] { share });

    var result = (await _sut.GetSharedDocsAsync(10)).ToList();

    Assert.Single(result);
    Assert.Equal(1, result[0].ShareId);
}
```

```
// — GetSharedDoc
```

```
[Fact]
public async Task GetSharedDoc_NotAccessible_ThrowsNotFoundException()
{
    _shares.Setup(s => s.GetByUserAndDocumentWithDocumentAsync(10, 99))
        .ReturnsAsync((UserDocument?)null);

    await Assert.ThrowsAsync<NotFoundException>(
        () => _sut.GetSharedDocAsync(10, 99));
}
```

```
[Fact]
public async Task GetSharedDoc_Accessible_ReturnsDetail()
{
    var share = MakeShare(10, 1, SharePermission.Owner);
    _shares.Setup(s => s.GetByUserAndDocumentWithDocumentAsync(10, 1))
        .ReturnsAsync(share);

    var result = await _sut.GetSharedDocAsync(10, 1);

    Assert.Equal(1, result.ShareId);
    Assert.Equal("Owner", result.Permission);
}
```

```
// — CreateDocument
```

```
[Fact]
public async Task CreateDocument_EmptyTitle_ThrowsValidationException()
{
    await Assert.ThrowsAsync<ValidationException>(
        () => _sut.CreateDocumentAsync(10, new CreateDocumentRequest("",
null)));
}
```

```
[Fact]
public async Task CreateDocument_Valid_PersistsDocumentAndOwnerShare()
{
    var loadedShare = MakeShare(10, 0, SharePermission.Owner);
```



```

        _shares.Setup(s => s.GetByUserAndDocumentWithDocumentAsync(10,
            It.IsAny<int>()))
            .ReturnsAsync(loadedShare);

        await _sut.CreateDocumentAsync(10, new CreateDocumentRequest("My Doc",
            "content"));

        _docs.Verify(d => d.AddAsync(It.Is<Document>(doc =>
            doc.Title == "My Doc" && doc.OwnerId == 10 && doc.Version == 1)),
            Times.Once);

        _shares.Verify(s => s.AddAsync(It.Is<UserDocument>(ud =>
            ud.UserId == 10 && ud.PermissionLevel == SharePermission.Owner)),
            Times.Once);

        _uow.Verify(u => u.CompleteAsync(), Times.Exactly(2));
    }

    // — UpdateDocument

```

```

    [Fact]
    public async Task UpdateDocument_EmptyContent_ThrowsValidationException()
    {
        await Assert.ThrowsAsync<ValidationException>(
            () => _sut.UpdateDocumentAsync(10, 1, new UpdateDocumentRequest("",
1))) );
    }

    [Fact]
    public async Task
UpdateDocument_ReadOnlyPermission_ThrowsForbiddenException()
    {
        var share = MakeShare(10, 1, SharePermission.ReadOnly);
        _shares.Setup(s => s.GetByUserAndDocumentWithDocumentAsync(10,
1)).ReturnsAsync(share);

        await Assert.ThrowsAsync<ForbiddenException>(
            () => _sut.UpdateDocumentAsync(10, 1, new UpdateDocumentRequest("new
content", 1)));
    }

    [Fact]
    public async Task UpdateDocument_VersionMismatch_ThrowsConflictException()
    {
        var doc = MakeDocument(); // version = 1
        var share = MakeShare(10, 1, SharePermission.ReadWrite, doc);
        _shares.Setup(s => s.GetByUserAndDocumentWithDocumentAsync(10,
1)).ReturnsAsync(share);

        // client sends version 2 but server is at 1
        await Assert.ThrowsAsync<ConflictException>(
            () => _sut.UpdateDocumentAsync(10, 1, new UpdateDocumentRequest("new
content", 2)));
    }

    [Fact]

```

```

public async Task UpdateDocument_Valid_IncrementsVersion()
{
    var doc = MakeDocument();
    var share = MakeShare(10, 1, SharePermission.ReadWrite, doc);
    _shares.Setup(s => s.GetByUserAndDocumentWithDocumentAsync(10,
1)).ReturnsAsync(share);

    await _sut.UpdateDocumentAsync(10, 1, new
UpdateDocumentRequest("updated", 1));

    Assert.Equal("updated", doc.Content);
    Assert.Equal(2, doc.Version);
    _docs.Verify(d => d.Update(doc), Times.Once);
}

// — DeleteSharedDoc

```

```

[Fact]
public async Task DeleteSharedDoc_AsOwner_DeletesAllSharesAndDocument()
{
    var doc = MakeDocument();
    var ownerShare = MakeShare(10, 1, SharePermission.Owner, doc);
    var otherShare = MakeShare(20, 1, SharePermission.ReadOnly, doc);

    _shares.Setup(s => s.GetByUserAndDocumentWithDocumentAsync(10,
1)).ReturnsAsync(ownerShare);
    _shares.Setup(s => s.GetAllByDocumentIdAsync(1)).ReturnsAsync(new[] {
ownerShare, otherShare });

    await _sut.DeleteSharedDocAsync(10, 1);

    _shares.Verify(s => s.Remove(ownerShare), Times.Once);
    _shares.Verify(s => s.Remove(otherShare), Times.Once);
    _docs.Verify(d => d.Remove(doc), Times.Once);
}

[Fact]
public async Task DeleteSharedDoc_AsNonOwner_RemovesOnlyCallerShare()
{
    var doc = MakeDocument();
    var share = MakeShare(20, 1, SharePermission.ReadWrite, doc);
    _shares.Setup(s => s.GetByUserAndDocumentWithDocumentAsync(20,
1)).ReturnsAsync(share);

    await _sut.DeleteSharedDocAsync(20, 1);

    _shares.Verify(s => s.Remove(share), Times.Once);
    _docs.Verify(d => d.Remove(It.IsAny<Document>()), Times.Never);
}

// — GetShares

```

```

[Fact]
public async Task GetShares_AsNonOwner_ThrowsForbiddenException()
{

```

```

        var share = MakeShare(10, 1, SharePermission.ReadOnly);
        _shares.Setup(s => s.GetByUserAndDocumentWithDocumentAsync(10,
1)).ReturnsAsync(share);

        await Assert.ThrowsAsync<ForbiddenException>(
            () => _sut.GetSharesAsync(10, 1));
    }

    [Fact]
    public async Task GetShares_AsOwner_ExcludesOwnerFromList()
    {
        var doc = MakeDocument();
        var ownerShare = MakeShare(10, 1, SharePermission.Owner, doc);
        var readerShare = MakeShare(20, 1, SharePermission.ReadOnly, doc);

        _shares.Setup(s => s.GetByUserAndDocumentWithDocumentAsync(10,
1)).ReturnsAsync(ownerShare);
        _shares.Setup(s => s.GetAllByDocumentIdAsync(1)).ReturnsAsync(new[] {
ownerShare, readerShare });
        _users.Setup(u => u.GetByIdAsync(20)).ReturnsAsync(new User { Id = 20,
Username = "bob" });

        var result = (await _sut.GetSharesAsync(10, 1)).ToList();

        Assert.Single(result);
        Assert.Equal(20, result[0].UserId);
    }

    // — AddShare

```

```

    [Fact]
    public async Task AddShare_AsNonOwner_ThrowsForbiddenException()
    {
        var share = MakeShare(10, 1, SharePermission.ReadWrite);
        _shares.Setup(s => s.GetByUserAndDocumentWithDocumentAsync(10,
1)).ReturnsAsync(share);

        await Assert.ThrowsAsync<ForbiddenException>(
            () => _sut.AddShareAsync(10, 1, new CreateShareRequest(20,
"ReadOnly")));
    }

    [Fact]
    public async Task AddShare_AlreadyShared_ThrowsConflictException()
    {
        var ownerShare = MakeShare(10, 1, SharePermission.Owner);
        _shares.Setup(s => s.GetByUserAndDocumentWithDocumentAsync(10,
1)).ReturnsAsync(ownerShare);
        _users.Setup(u => u.GetByIdAsync(20)).ReturnsAsync(new User { Id = 20
});
        _shares.Setup(s => s.GetByUserAndDocumentAsync(20, 1))
            .ReturnsAsync(MakeShare(20, 1, SharePermission.ReadOnly));

        await Assert.ThrowsAsync<ConflictException>(
            () => _sut.AddShareAsync(10, 1, new CreateShareRequest(20,
"ReadOnly")));
    }

```

```

    }

    [Fact]
    public async Task AddShare_InvalidPermission_ThrowsValidationException()
    {
        var ownerShare = MakeShare(10, 1, SharePermission.Owner);
        _shares.Setup(s => s.GetByUserAndDocumentWithDocumentAsync(10,
1)) .ReturnsAsync(ownerShare);
        _users.Setup(u => u.GetByIdAsync(20)) .ReturnsAsync(new User { Id = 20
});
        _shares.Setup(s => s.GetByUserAndDocumentAsync(20,
1)) .ReturnsAsync((UserDocument?)null);

        await Assert.ThrowsAsync<ValidationException>(
            () => _sut.AddShareAsync(10, 1, new CreateShareRequest(20,
"Admin"))) );
    }

    // — RevokeShare

```

```

    [Fact]
    public async Task RevokeShare_AsOwner_RemovesTargetShare()
    {
        var ownerShare = MakeShare(10, 1, SharePermission.Owner);
        var targetShare = MakeShare(20, 1, SharePermission.ReadOnly);
        _shares.Setup(s => s.GetByUserAndDocumentWithDocumentAsync(10,
1)) .ReturnsAsync(ownerShare);
        _shares.Setup(s => s.GetByUserAndDocumentAsync(20,
1)) .ReturnsAsync(targetShare);

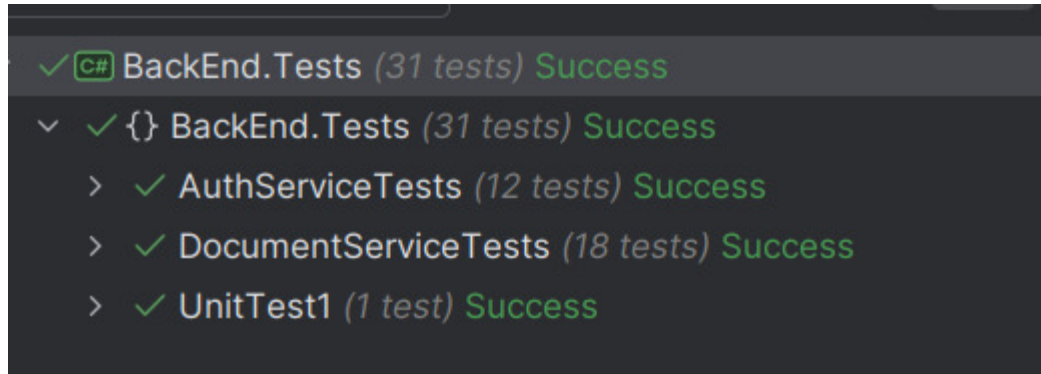
        await _sut.RevokeShareAsync(10, 1, 20);

        _shares.Verify(s => s.Remove(targetShare), Times.Once);
        _uow.Verify(u => u.CompleteAsync(), Times.Once);
    }

    [Fact]
    public async Task RevokeShare_TargetNotFound_ThrowsNotFoundException()
    {
        var ownerShare = MakeShare(10, 1, SharePermission.Owner);
        _shares.Setup(s => s.GetByUserAndDocumentWithDocumentAsync(10,
1)) .ReturnsAsync(ownerShare);
        _shares.Setup(s => s.GetByUserAndDocumentAsync(99,
1)) .ReturnsAsync((UserDocument?)null);

        await Assert.ThrowsAsync<NotFoundException>(
            () => _sut.RevokeShareAsync(10, 1, 99));
    }
}

```



Appendix D: Shared Tasks

- **Lionte Eduard-Iulian**
 - Functionality of the interface
 - Api requests to backend
- **Gorea Sabin-Gabriel**
 - Api Controllers
 - JWT and hashing
- **Mârț Sebastian**
 - Database Interaction
 - Documentation
- **Miron Victor**
 - User Interfaces
 - HelpDoc

